

Best CS 340
pickup line?



CS 340

Storing Data

prof, Legende

clicker.cs.illinois.edu

Q1

~Code~
340



Big thanks
to Prof Wade
Fagen-Ulmschneider
for the free
clicker!

Updates

90% wed

1. MP 1.1, MP 1.2, MP 1.3, environment check-off

Today

1. HW 3 - wed

1. MP 2 - 2 weeks tues

PN 6

1. Canvas grades ←

1. Exam 1

← next week Thurs.
no class ~~thurs~~ next thurs

study guide
practice exam
by this weekend

Would you attend an MPP? Machine Problem - 2 Party?

Thursday
after class
in Siebel
4th floor

3:30-4:30 ish

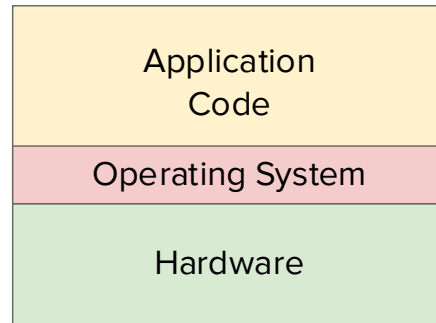
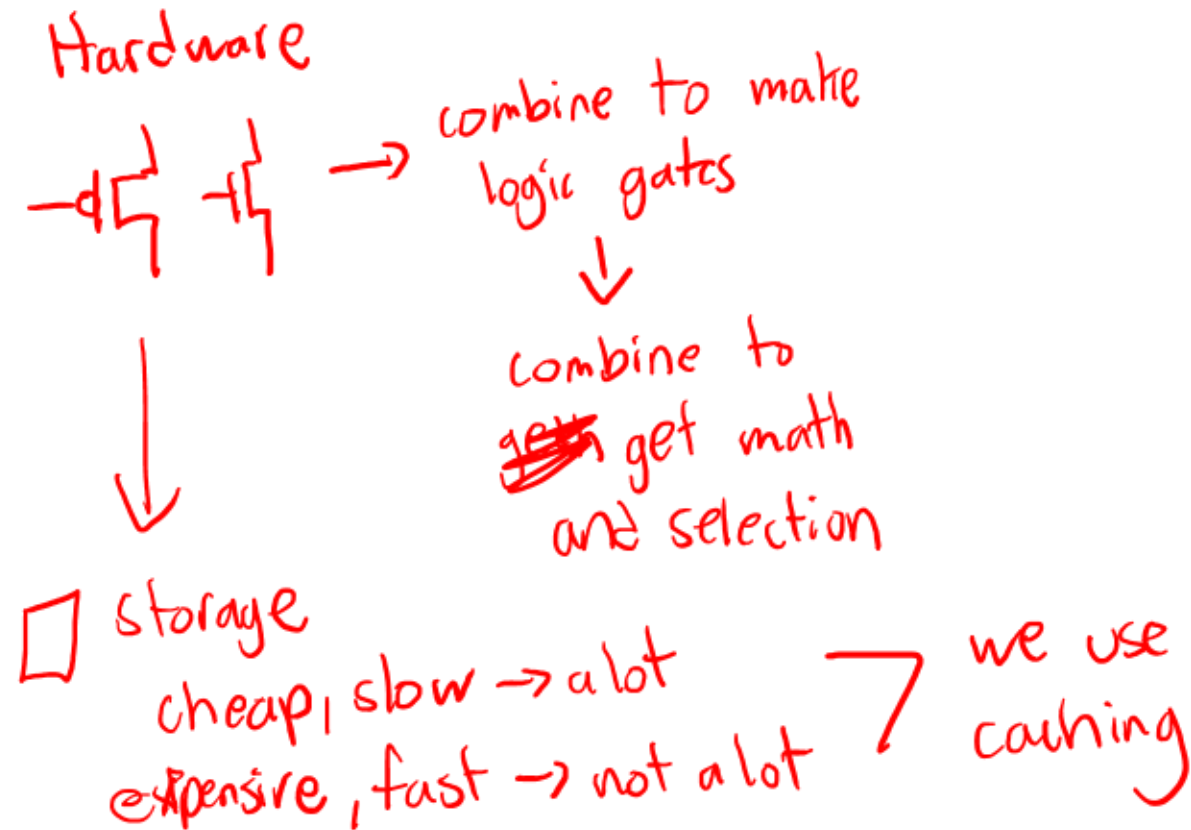
clicker.cs.illinois.edu

Q2

~Code~
340



Big Picture



Caching - an algorithm for utilizing fast small memory and slow big memory.

Locality - the idea that computers often use nearby and similar information sequentially.

Spatial locality - things nearby are used more likely

Temporal locality - things used recently more likely

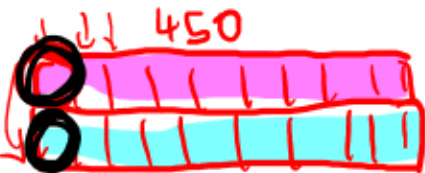
How can you change this code for better locality?

```
6   int arr1[500];  
7   int arr2[500];  
8   //add stuff to arrays  
9   int count = 0;  
10  for(int i = 0 ; i < 500; i++){  
11  | → if(arr1[i]%2 == 0) count++;  
12  | → if(arr2[i]%2 == 0) count++;  
13  | }  
    }
```

for (...) {
 if (arr2[i] == ...) {
 }
}

How can you change this code for better locality?

```
8   int doub[500][450];  
9   //add stuff to doub  
10  for(int col = 0; col < 450; col++){  
11      for(int row = 0; row < 500; row++){  
12          doub[row][col]++;  
13      }  
14  }
```



500 ↓

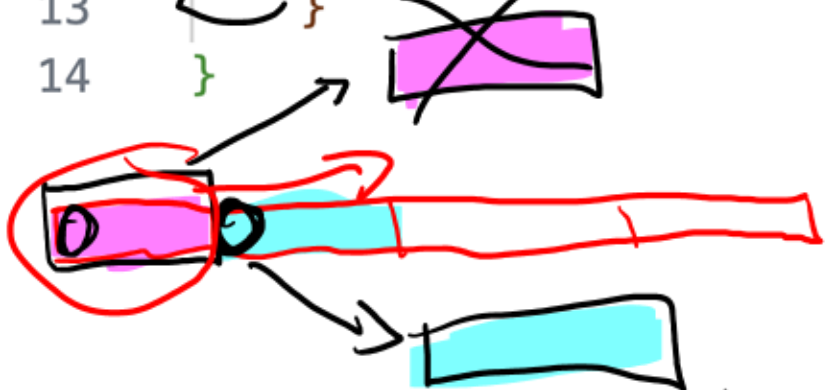
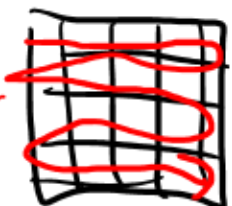


col=0
row=0

col=0
row=1

for(int row...

for(int col...



Storing Data

Today's LGs - Building up your intuition when working with bytes.

- Be able to describe large numbers in bytes.
- Be able to describe small numbers in bytes.
 - Be able to go between hexadecimal, decimal, and binary.
 - Be able to go between little and big endianness.
- Understand the implications of how bytes are stored when coding in C.

Agenda

1. Bytes
2. Data Types (endianness)

**How to represent
things with high
and low voltages**

How do we talk about small amounts of data?



high, low



1, 0

binary

$$\begin{array}{ccc} 1 & 1 & 0 \\ \uparrow & \uparrow & \uparrow \\ 2^2 & 2^1 & 2^0 = 6 \end{array}$$

working with 1 bit
at a time is too small

Bytes



Idea 1 - Bytes are a unit indicating 8 bits (1's and 0's)

Idea 2 - The base-2 (binary) number system is a way of interpreting 1's and 0's to represent amounts

10001 - red

01001 - blue

0b0000 0110

6 in byte form

0006

How do humans talk about bytes?

10101 10001010010

issue - binary is hard to read

potential idea 1 - write out the value
of the byte in decimal

What is the biggest value in decimal we can represent with 1 byte?

0b1111 1111 $\leftarrow ? = 255$

25 13 (207) (16) $\leftarrow 2$...

problem $\uparrow$ 3 symbols
- hard to go back to binary

clicker.cs.illinois.edu

Q3

~Code~
340



- A) 127
- B) 128
- C) 255
- D) 256

Issue - Binary is hard to read/write for humans

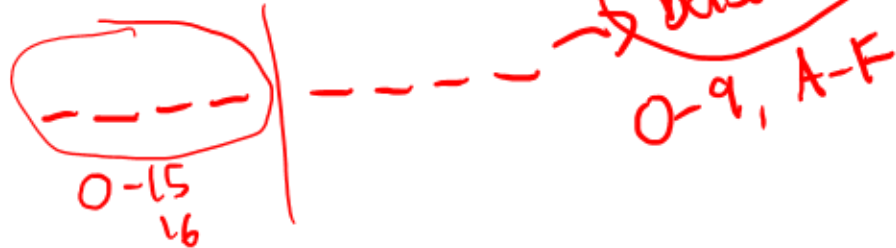
1. Base-10

- doesn't work well going back & forth
- 3 digits

2. ~~256~~ each byte is 1 digit - base-256

- need 256 unique symbols

3. 2 digits per byte



Hexadecimal

Base-16 number system

↓
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ¹⁰A, ¹¹B, ¹²C, ¹³D, ¹⁴E, ¹⁵F

Ox - header for hex

$$\begin{array}{ccc} \text{hex} & \text{dec} & \text{bin} \\ \text{Ox} \text{0F} & = 15 & = 0000 1111 \end{array}$$

$$\text{Ox} \text{10} = 16 = 0001 0000$$

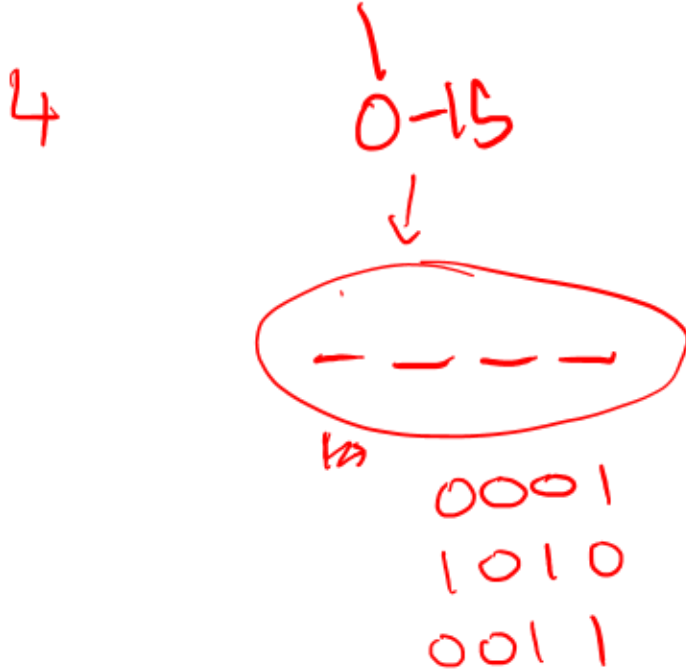
$$\text{Ox} \text{13} = 19 = 0001 0011$$

⇓ - easy to convert to binary

⇓ - 2 digits per byte

⇓ - new number system to learn

How many bits can 1 digit in hexadecimal represent?



clicker.cs.illinois.edu

Q4

~Code~
340



- A) 1
- B) 2
- C) 4
- D) 8

How many digits of hexadecimal are needed to represent 1 byte?

1-byte = 8 bits

4 bits = 1 hex digit

1 byte = 2 hex digits

clicker.cs.illinois.edu

Q5

~Code~
340



- A) 1
- B) 2
- C) 3
- D) 4

Big Picture Review

Idea 1 - A byte is a unit indicating 8 bits (1's and 0's).

Idea 2 - The base-2 (binary) number system is a way of interpreting 1's and 0's to represent bigger numbers

Idea 3 - To make bytes easier to work with we represent the value the bits hold in hexadecimal.

Idea 4 - Hexadecimal is easy to convert to binary and back.

What is 0x2A in binary?

0 1 2 3 4 5 6 7 8 9 A B C D E F



01000101010

What is 0x104F in binary?

- A) 0b0001 0000 0100 1111
- B) 0b0001 0000 1111 0100
- C) 0b0001 0000 0010 1111
- D) 0b0001 0000 1111 0010

clicker.cs.illinois.edu

Q6

~Code~
340



What is 0b1011 1111 in hexadecimal?



- A) 0x10F
- B) 26
- C) 0x11F
- D) 0xBF

**What is 225 in
hexadecimal?**

2-MUX Review

~~the~~ more depth == slower

Scales by powers of 2

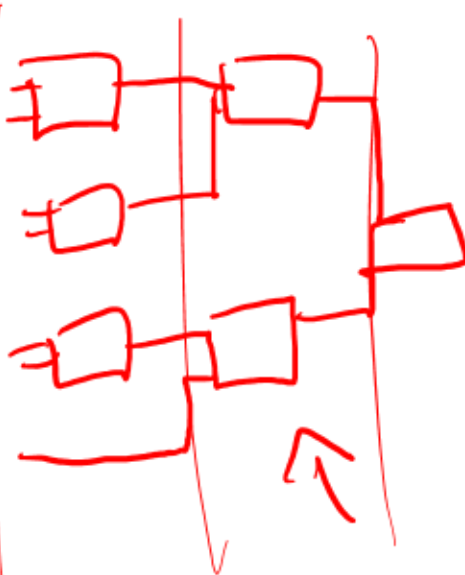
$[0, 1, 2, 3, 4, 5, 6, 7]$

8 selection
depth 3



$[0, 1, 2, 3, 4, 5, 6]$

7 selection
depth 3



How do we talk about big numbers?

My computer's memory holds... **64 GB**

Value	base-10	Prefix	Pronounced
2^{10}	1024	Ki ✓	Kilo
2^{20}	1,048,576	Mi	Mega
2^{30}	1,073,741,824	Gi	Giga
2^{40}	1,099,511,627,776	Ti	Tera
2^{50}	1,125,899,906,842,624	Pi	Peta
2^{60}	1,152,921,504,606,846,976	Ei	Exa

* don't need to memorize

2^6 GiB

$$2^6 \cdot 2^{30} = 2^{36}$$

$$2^{54} = 2^{50} \cdot 2^4 = 16 \text{ PiB}$$

What is the following translated to?

$2^{(45)}$ Bytes

$$2^{40} \cdot 2^5 = 32$$

Value	base-10	Prefix	Pronounced
2^{10}	1024	Ki	Kilo
2^{20}	1,048,576	Mi	Mega
2^{30}	1,073,741,824	Gi	Giga
2^{40}	1,099,511,627,776	Ti	Tera
2^{50}	1,125,899,906,842,624	Pi	Peta
2^{60}	1,152,921,504,606,846,976	Ei	Exa

clicker.cs.illinois.edu

Q8

~Code~
340



~~A) 16 GB~~

~~B) 32 GB~~

C) 16 TB

D) 32 TB

E) 16 PB

Data Types

char - 1 byte



- Can print out as an ascii character
- Can hold 1 byte - 8 bits

Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
00	NUL	10	DLE	20	SP	30	0	40	@	50	P	60	`	70	p
01	SOH	11	DC1	21	!	31	1	41	A	51	Q	61	a	71	q
02	STX	12	DC2	22	"	32	2	42	B	52	R	62	b	72	r
03	ETX	13	DC3	23	#	33	3	43	C	53	S	63	c	73	s
04	EOT	14	DC4	24	\$	34	4	44	D	54	T	64	d	74	t
05	ENQ	15	NAK	25	%	35	5	45	E	55	U	65	e	75	u
06	ACK	16	SYN	26	&	36	6	46	F	56	V	66	f	76	v
07	BEL	17	ETB	27	'	37	7	47	G	57	W	67	g	77	w
08	BS	18	CAN	28	(	38	8	48	H	58	X	68	h	78	x
09	HT	19	EM	29	)	39	9	49	I	59	Y	69	i	79	y
0A	LF	1A	SUB	2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
0B	VT	1B	ESC	2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
0C	FF	1C	FS	2C	,	3C	<	4C	L	5C	\	6C	l	7C	
0D	CR	1D	GS	2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
0E	SO	1E	RS	2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
0F	SI	1F	US	2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

A char holds 1 byte, what's the biggest value it can hold in hexadecimal?

$0xF = 1111$ ~~1111~~
00

$0xFF = 0b11111111$

clicker.cs.illinois.edu

Q9

~Code~
340



A) 0xF0

B) 0xFF

~~C) 0b11111111~~

D) 0xFFFF

Tying it all together in C

char var = 'J' → 74 = 0x4A
 hardware ↓ ↓
 0100 1010



```
printf("%d", var); → 74
```

```
printf("%x", var); 0x4A
```

Data is stored as bytes

main.c



```
1  #include <stdio.h>
2
3  int main() {
4      printf("Hello World\n");
5      return 0;
6  }
```

main.c



23 69 6E 63 6C 75 64 65 20 3C 73 74 64 69 6F 2E
68 3E 0A 0A 69 6E 74 20 6D 61 69 6E 28 29 20 7B
0A 20 20 20 70 72 69 6E 74 66 28 22 48 65 6C 6C
6F 20 57 6F 72 6C 64 5C 6E 22 29 3B 0A 20 20 20
72 65 74 75 72 6E 20 30 3B 0A 7D 0A 0A 0A 0A
0A 0A 0A +



What is 0x23 in ASCII?

- A) #
- B) i
- C) 23
- D) I need an ascii table please

clicker.cs.illinois.edu

Q10

~Code~
340



```
1  #include <stdio.h>
```

```
2
```

```
3  int main() {
```

```
4      printf("Hello World\n");
```

```
5      return 0;
```

```
6  }
```

```
23 69 6E 63 6C 75 64 65 20 3C 73 74 64 69 6F 2E
68 3E 0A 0A 69 6E 74 20 6D 61 69 6E 28 29 20 7B
0A 20 20 20 70 72 69 6E 74 66 28 22 48 65 6C 6C
6F 20 57 6F 72 6C 64 5C 6E 22 29 3B 0A 20 20 20
72 65 74 75 72 6E 20 30 3B 0A 7D 0A 0A 0A 0A
0A 0A 0A +
```

main.c

Compiling a C program

main.c in hex view

```
23 69 6E 63 6C 75 64 65 20 3C 73 74 64 69 6F 2E
68 3E 0A 0A 69 6E 74 20 6D 61 69 6E 28 29 20 7B
0A 20 20 20 70 72 69 6E 74 66 28 22 48 65 6C 6C
6F 20 57 6F 72 6C 64 5C 6E 22 29 3B 0A 20 20 20
72 65 74 75 72 6E 20 30 3B 0A 7D 0A 0A 0A 0A
0A 0A 0A +
```

ascii

a.out

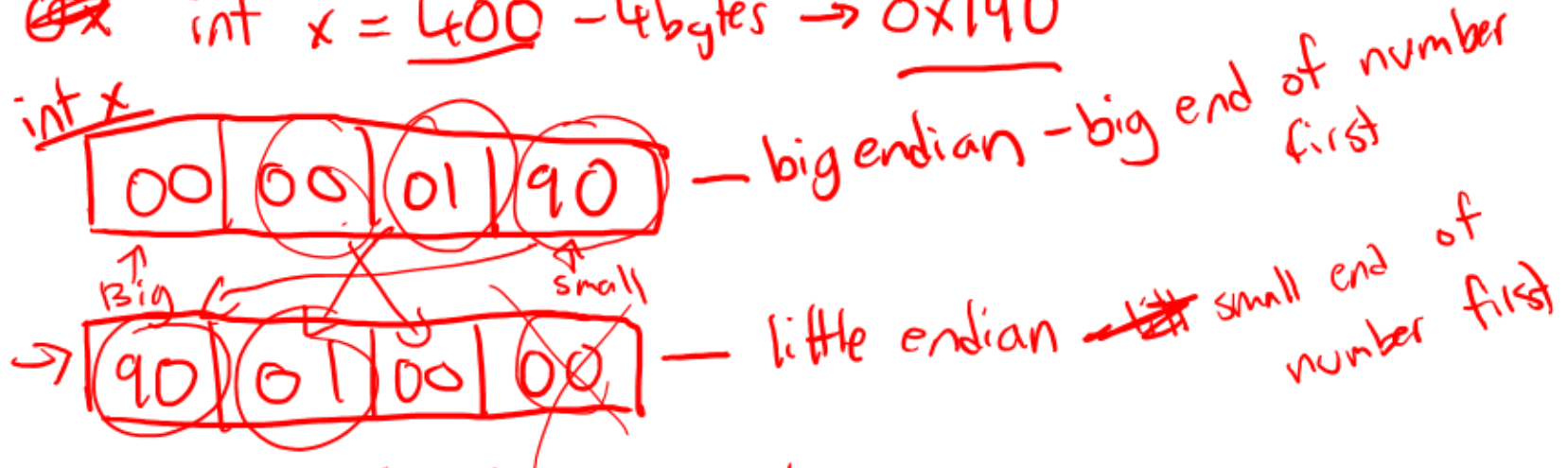
7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00
02 00 B7 00 01 00 00 00 40 05 40 00 00 00 00 00
40 00 00 00 00 00 00 00 F8 0B 01 00 00 00 00 00
00 00 00 00 40 00 38 00 09 00 40 00 1C 00 1B 00
06 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
40 00 40 00 00 00 00 00 40 00 40 00 00 00 00 00
F8 01 00 00 00 00 00 00 F8 01 00 00 00 00 00 00
08 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00
38 02 00 00 00 00 00 00 38 02 40 00 00 00 00 00
38 02 40 00 00 00 00 00 1B 00 00 00 00 00 00 00
1B 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00
01 00 00 00 05 00 00 00 00 00 00 00 00 00 00 00
00 00 40 00 00 00 00 00 00 00 40 00 00 00 00 00
BC 07 00 00 00 00 00 00 BC 07 00 00 00 00 00 00
00 00 01 00 00 00 00 00 01 00 00 00 06 00 00 00
E8 FD 00 00 00 00 00 00 E8 FD 41 00 00 00 00 00

Int - 4 bytes - 32 1's and 0's

- Can print out as negative or positive number (more on this later)
- Can hold any value represented by 32 bits (1's and 0's)

Little and Big Endian - the order the bytes are stored in multi-byte objects

~~int~~ int x = 400 - 4 bytes $\rightarrow$ 0x190



network - big endian

computer - little endian

Little and Big Endian - the order the bytes are stored in multi-byte objects

want to store value $0xFOAB1100$ in an int
403742848

- computer stores it as little endian



- you print that int

↓
will print $FOAB1100$

char - 0xFA ~ not matter

little

FA

big

FA

int - 0xAB000110

little

10	01	00	AB
----	----	----	----

big

AB	00	01	10
----	----	----	----

int arr[2] = {5, 899};

↓ ↓
0x05 0x0383

5

899



little

big

00 00 00 05 | 00 00 03 83

char arr[3] = "JJ";

0x4A



With which of the following types do we need to consider endianness?

~~A) char - 1 byte~~

B) char* - 8 bytes

C) int - 4 bytes

~~D) array of chars -~~

E) array of ints

~~F) None of the above~~

clicker.cs.illinois.edu

Q11

~Code~
340



Big Ideas

store's things in 1's and 0's

hex is used to read 1's and 0's

dec is used to make hex more readable

Endianness and Memory Layout

[View...](#)

Below we show ten bytes of little-endian memory at several addresses, using 2-hex-digit representations of each byte.

→ Address	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
→ Value	A5	D3	AC	D6	77	2A	37	3B	30	46

Suppose a uint16_t *p (i.e. a pointer to unsigned 16-bit integers) has value p = 1005.

What is the value of $*(p - 1)$? Answer in hexadecimal.

→ 2 bytes

*(p - 1) ?

[Save & Grade](#)[Save only](#)[New variant](#)

1005 - 2 bytes
1003

Endianness and Memory Layout

[View...](#)

Below we show ten bytes of **little-endian memory** at several addresses, using 2-hex-digit representations of each byte.

Address	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
Value	F6	DC	6D	CD	58	AF	22	49	BC	E3

Suppose a `uint16_t *p` (i.e. a pointer to unsigned 16-bit integers) has value `p = 1006`.

What is the value of `p[1]`? Answer in hexadecimal.

`p[1]`

integer in base 16



Save & Grade

Save only

clicker.cs.illinois.edu

Q12

~Code~
340



Endianness and Memory Layout

[View...](#)

Below we show ten bytes of **big-endian memory** at several addresses, using 2-hex-digit representations of each byte.

Address	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
Value	D5	14	F2	07	B3	4E	3A	C4	BD	2C

Suppose a `uint16_t *p` (i.e. a pointer to unsigned 16-bit integers) has value `p = 1006`.

What is the value of `*(p - 1)`? Answer in hexadecimal.

`*(p - 1)`

integer in base 16



Save & Grade

Save only

clicker.cs.illinois.edu

Q13

~Code~
340



Challenge: Does this computer use little or big endian?

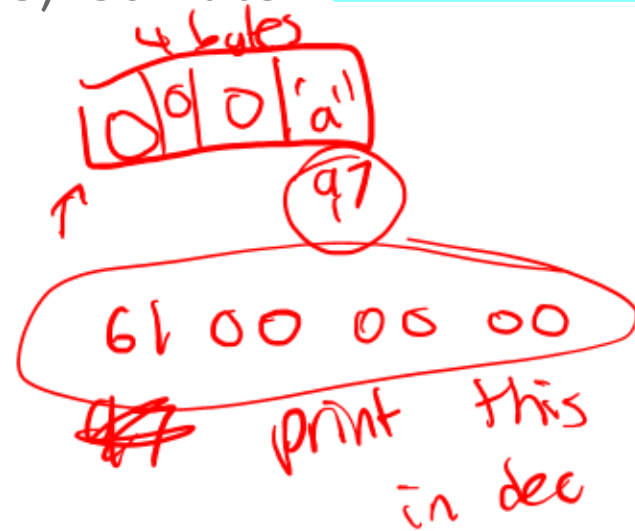
clicker.cs.illinois.edu

Q14

~Code~
340



- A) Little
- B) Big
- C) Can't tell



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      char *ptr = calloc(1, 4);
6      ptr[3] = 'a'; //ascii 97
7      → int *ptr2 = (int*)ptr;
8      printf("%i\n", *ptr2);
9  }
```

PROBLEMS

OUTPUT

TERMINAL 2

✓ TERMINAL

root@7abba03ac098: /workspaces/bst# ./a.out
1627389952

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      char *ptr = calloc(1, 4);
6      ptr[0] = 'a'; //ascii 97
7      int *ptr2 = (int*)ptr;
8      printf("%i\n", *ptr2);
9  }
```

PROBLEMS

OUTPUT

TERMINAL 2

✓ **TERMINAL**

● root@7a6ba03ac098:/workspaces/bst# ./a.out
97

**Why we don't use base-15 or base-17
instead of base-16?**